Colorado State University Pueblo

School of Engineering

# Hybrid Physics-Transformer Surrogate Modeling for Train System Dynamics

George James

April 23, 2026

# Abstract

This thesis presents a hybrid physics-transformer surrogate modeling framework for train system dynamics. The framework combines reduced-order physics models with transformer-based neural networks to enable efficient probabilistic forecasting of energy consumption, arrival times, and safety-critical outputs such as coupler forces. The approach addresses the computational limitations of high-fidelity simulation while maintaining physical plausibility and providing calibrated uncertainty estimates. Validation demonstrates improved accuracy and calibration compared to physics-only baselines across diverse route profiles, consist configurations, and control strategies.

# Acknowledgments

# Contents


**Abstract** **3**

**Acknowledgments** **5**

**1 Introduction and Problem Motivation** **13**
1.1 Background: Longitudinal Train Dynamics . . . . . 13
1.2 Surrogate Modeling . . . . . 13
1.3 Thesis Objective and Contributions . . . . . 14
1.4 Organization of Thesis . . . . . 14

**2 Related Work** **15**
2.1 Classical Train Longitudinal Dynamics and Simulation . . . . . 15
2.1.1 Modeling Abstractions and Levels of Fidelity . . . . . 15
2.1.2 Computational Schemes and Numerical Challenges . . . . . 16
2.1.3 Benchmarking, Applications, and Limitations . . . . . 16
2.2 Data-Driven and Hybrid Modeling in Rail . . . . . 17
2.2.1 Surrogate Models for Accelerating Physics-Based Simulation . . . . . 17
2.2.2 Data-Driven Estimation of In-Train Forces . . . . . 18
2.2.3 Surrogate Modeling for Operations and Optimization . . . . . 18
2.2.4 Limitations of Existing Data-Driven and Hybrid Approaches . . . . . 18
2.2.5 Implications for Hybrid Probabilistic Modeling . . . . . 19
2.3 Transformers for Dynamical Systems and Spatiotemporal Forecasting . . . . 19
2.3.1 Long-horizon Forecasting Transformers . . . . . 19
2.3.2 Neural Operators for PDE-governed Dynamics . . . . . 20
2.3.3 Implications for Train Dynamics and Spatiotemporal Decision Support 21
2.4 Probabilistic Forecasting and Uncertainty Calibration . . . . . 22
2.4.1 Foundations of Probabilistic Regression . . . . . 23
2.4.2 Empirical Miscalibration in Deep Models . . . . . 24
2.4.3 Post-hoc Calibration for Regression . . . . . 25
2.4.4 Distribution-Free Coverage via Conformal Prediction . . . . . 26
2.4.5 Proper Scoring Rules and Evaluation Metrics . . . . . 27

**3 Problem Formulation** **29**
3.1 Forecasting problem, classical dynamics, and tensorized formulation . . . . . 29
3.1.1 Physical setting and prediction task . . . . . 29

3.1.2 Classical longitudinal dynamics formulation . . . 30
3.1.3 Tensorized state representation . . . 30
3.1.4 Dynamics in tensor-oriented form . . . 31
3.1.5 Forecasting objective and simulator operator . . . 31
3.2 Reference longitudinal train dynamics model . . . 32
3.2.1 Stage 1: single powered vehicle . . . 32
3.2.2 Stage 2: locomotive and one car with a linear coupler . . . 33
3.2.3 Stage 3: nonlinear coupler with slack and asymmetric buff/draft . . . 34
3.2.4 Stage 4: three bodies and sequential coupler forces . . . 36
3.2.5 Stage 5: general $N$-vehicle assembly . . . 36
3.2.6 Stage 6: per-vehicle grade and curvature from a route profile . . . 36
3.2.7 Stage 7: actuator dynamics and power-limited traction . . . 40
3.2.8 Stage 8: scope relative to industrial high fidelity . . . 41
3.3 Input definitions . . . 41
3.3.1 Route profile . . . 41
3.3.2 Consist configuration . . . 41
3.3.3 Control plan . . . 41
3.4 Output definitions . . . 42
3.4.1 Trajectory-level outputs . . . 42
3.4.2 Aggregate outputs . . . 42
3.4.3 Risk metrics . . . 42
3.5 Uncertainty sources . . . 42
3.5.1 Formal Treatment of Uncertainty . . . 43
3.6 Supervised Learning Objective . . . 43
3.6.1 Loss Term Definitions . . . 43
3.7 Conceptual Pipeline . . . 44
3.8 Evaluation Targets . . . 44
3.9 Chapter summary . . . 44

# List of Figures


2.1 Quantile calibration: diagonal = perfect; uncalibrated model deviates; isotonic calibration corrects. . . . . 26

3.1 Stage 1 reference run: single vehicle on level track with a smooth traction ramp. Top to bottom: position, speed, and net longitudinal force $F_{\text{trac}} - R(v)$ versus time. . . . . 33
3.2 Stage 2: two-body train with linear coupler. Speeds of lead and trailing vehicle (overlapping at this scale), coupler force on the rear car, and extension $\delta$. . . 34
3.3 Stage 3: comparison of linear spring–damper force (Stage 2 parameters) versus nonlinear slack coupler force along the same trajectory, with extension and relative speed. . . . . 35
3.4 Stage 4: three-body train with nonlinear couplers. Speeds, coupler forces $F_0$ and $F_1$, and end-to-end stretch versus time. . . . . 37
3.5 Stage 4 optional scenario: lead-only brake pulse; speeds and coupler forces. . 38
3.6 Stage 5: $N$-vehicle train; selected speeds and coupler forces versus time. . . . 38
3.7 Stage 5: coupler force heatmap (force versus time and coupler index). . . . . 39
3.8 Stage 6: interpolated grade profile $\sin\theta$ versus route position, with lead and rear speeds and coupler forces versus simulation time. . . . . 39
3.9 Stage 7: lead speed with brake-command onset marker; first-order lag states $z_{\text{brk}}$ and $z_{\text{trac}}$ on the lead versus time. . . . . 40
3.10 Conceptual pipeline from structured inputs to probabilistic outputs via high-fidelity simulation and surrogate modeling. . . . . 44

# List of Tables

1.1 Representative computational cost of physics-based simulations . . . . . 13

2.1 Representative longitudinal train dynamics (LTD) simulation tools . . . . . 17

# Chapter 1

# Introduction and Problem Motivation

## 1.1 Background: Longitudinal Train Dynamics

Longitudinal train dynamics (LTD) concern the motion and force distribution through the length of a train. A freight train is a distributed mechanical system, often consisting of tens to hundreds of individual coupled masses. The system involves complex and highly non-linear interactions between locomotives, freight cars, and the track profile. Modern simulations of these system interactions rely on detailed physical models which require solving differential equations for each vehicle, resulting in high computational cost. Rapid scenario exploration can be impossible due to the computational resource demand.

| Model Type | CPU-hours | Source |
|---|---:|---|
| Reduced / Lumped Models | 0.05 | [1] |
| High-Fidelity Train Longitudinal Dynamics | 50 | [2] |
| High-Fidelity Multibody / Vehicle Dynamics | 500 | [3] |
| Transient CFD (Engineering Scale) | 50 000 | [4] |
| Extreme CFD / HPC Simulation | 275 000 | [5] |

Table 1.1: Representative computational cost of physics-based simulations

## 1.2 Surrogate Modeling

High-fidelity physics simulators are essential for understanding complex physical systems, however the computational expense of using such simulators can put serious speed limits on evaluation. This is the case especially in modern analysis or control problems, where complex simulations can require hundreds to thousands of hours of CPU time to simulate multi-body systems. These models are invaluable for theory validation, but they are completely impractical when there exists a need for rapidly repeated scenario evaluation.

Surrogate modeling addresses this problem by learning an approximate mapping of the inputs and system response outputs. This learned model retains the underlying physics at an approximated level, but more importantly can reduce evaluation cost by several orders

of magnitude. Trained surrogate models can typically output an accurate prediction in milliseconds or less. This enables rapid simulation of a complex system that would normally consume many CPU hours.

Surrogate models do not replace deterministic physics models, but rather complement them. High-fidelity simulations allow for the data generation needed to build a surrogate model, and also serve as the physical grounding which couples with the surrogate to produce a rapid yet physically informed simulation environment.

## 1.3 Thesis Objective and Contributions

This thesis presents a physics-transformer hybrid architecture that combines deterministic, reduced-order physics models of the longitudinal train dynamics with a data-driven transformer model. This architecture will enable rapid scenario evaluation while retaining physical constraints.

1. A modular schema that integrates physics-based feature generation with transformer-based scenario modeling
2. Probabilistic output formulation representing quantile regression and exceedance probability estimation
3. Validation methodology demonstrating improved accuracy and reduced computation load compared to physics-only baselines
4. Ablation studies analyzing the contribution of physics integration and attention factorization

## 1.4 Organization of Thesis

This thesis is organized as follows. Chapter 2 reviews related work in train dynamics, surrogate modeling, and transformer architectures. Chapter 3 formulates the problem mathematically, defining inputs, outputs, and uncertainty sources. **??** presents the framework architecture, including the physics layer, transformer surrogate, and hybrid integration strategy. **??** describes data generation and training methodology. **??** presents evaluation results and validation studies. **??** discusses implications and limitations. **??** concludes with future work directions.

# Chapter 2

# Related Work

## 2.1 Classical Train Longitudinal Dynamics and Simulation

Longitudinal train dynamics (LTD) is the study of the physical interactions within a connected train body comprised of multiple linked masses, henceforth referred to as a *consist*. Classical longitudinal train dynamics is defined as the motion of rolling stock vehicles in the longitudinal dimension of the track, ignoring lateral or vertical forces in most cases. The motion of the whole train as well as the individual motions of each vehicle are accounted for in this schema [6].

Early attempts at LTD study treated a train as a continuous elastic rod with a traction force applied at one end, and enabled the study of longitudinal vibration phenomena. This later progressed to discretized mass-spring chains. Early models such as these generally ignored discontinuities, such as those arising from coupler slack, which enabled closed form solutions that required no numerical solver. Nonlinearities were identified early, such as draft gear behavior, but could not be simulated with the level of computing power at the time [7].

### 2.1.1 Modeling Abstractions and Levels of Fidelity

LTD can be framed as layered abstractions with increasing levels of complexity. At the greatest order-reduction level, LTD can be modeled as point mass or a single-body train model. This level of complexity can be useful for modeling energy usage or scheduling, but is highly limited in the observability of the system, i.e. many variables are unaccounted for. At the next level of abstraction, the consist can be modeled as a multi-mass system, where each vehicle is accounted for as a rigid body with one longitudinal degree of freedom. Masses are connected with modeled couplers and draft gears, which introduce nonlinear interactions. Beyond this, we can extend the DOF of the model to account for motions in the remaining spatial dimensions, which allows for observability track curve effects, in train instabilities, and even derailment scenarios. This scale of modeling naturally trades speed for fidelity, as computation of so many parameters is highly expensive, but the high fidelity is necessary for some analyses [7].

### 2.1.2 Computational Schemes and Numerical Challenges

The standard computation schema for LTD treats the problem as an repeated evaluation of all force elements followed by state propagation [8]. In general, a simulator first identifies force elements (propulsion resistance, curving resistance, grade components, gravity, traction forces, dynamic brake forces, air brake forces, and coupler forces). Then, the system computes each element from the inputs (time, vehicle velocities, accelerations, and vehicle locations). Finally, velocities and positions are updated through a numerical solver. The clear modularity of this computing system allows for addition or replacement of new computing models, such as replacing a module with a learned surrogate model.

The critical difficulty with computing LTD is with the vehicle connection systems, specifically the couplers, draft gears, and drawbars. Friction draft gears exhibit extreme nonlinearities, including hysteresis and stick-slip) that are hard to both model and numerically solve [7]. The coupler force calculation accounted for approximately 65% of the computing time in the study described in [8]. The difficulty of handling this interaction further motivates the effectiveness of the hybrid physics and transformer surrogate model, which is focused on handling the hard parts of the model while keeping the backbone physics modeling intact.

### 2.1.3 Benchmarking, Applications, and Limitations

LTD has historically lacked standardized procedures for benchmarking LTD simulators, which limits objective comparison of tools developed by different institutions. Spiryagin et. al. introduced an international benchmarking initiative to define a mutual set of input scenarios, operating conditions, and necessary outputs to compare efficacy of various simulators [9]. The benchmarking initiative was motivated by the fact that most LTD simulation tools have evolved independently as in-house software for the many institutions that use them. This resulted in limited to no transparency in terms of numerical methods, assumptions, and performance. Participating simulators used identical train configurations, commands for traction and braking, and track profiles. Outputs were measured in high temporal resolution to enable detailed comparison. This work provides an indispensable reference point for evaluating simulation fidelity and computational performance. The work further motivates the need for reproducible, scenario-based evaluation systems.

The applications of LTD, especially rapidly evaluated scenarios, is further highlighted in the Wu-Spiryagin-Cole paper. Driving advisory systems are integrated computation engines which advise locomotive operators to take actions based on instrumented vehicle data. These systems require real time evaluation of LTD outputs. Furthermore, energy consumption calculations can be evaluated from a simulator with observable traction/braking forces and speed/distance signals. The accuracy of this energy consumption can be heightened with strong predictions for interactions that are harder to model (handled by the transformer module). Fatigue and safety analyses depend on time-domain histories of dynamic interactions, which can be invaluable for risk assessment.

Table 2.1: Representative longitudinal train dynamics (LTD) simulation tools

| Tool | Origin | Scope | Availability |
|---|---|---|---|
| **TrainDy** [10] | UIC-oriented (academic/industry consortium) | Freight LTD for operations and safety analysis | Specialized / consortium |
| **TOES™** [11] | Developed for AAR-member railroads | Comprehensive train dynamics for operating scenarios | Commercial / licensed |
| **TEDS** [12, 13] | U.S. FRA-sponsored (contractors) | Safety/risk evaluation, incident investigation, operations LTD | Publicly documented |
| **UM Train** [14] | Universal Mechanism suite | Train longitudinal dynamics within multibody ecosystem | Commercial |
| **TDEAS** [15] | Developed in China (research/industry) | LTD + energy analysis | Research / institutional |
| **CRE-LTS** [9] | Centre for Railway Engineering (CQU) | Benchmarking-ready research LTD simulator | Research / in-house |

## 2.2 Data-Driven and Hybrid Modeling in Rail

The computational expense and architectural complexity of high-fidelity rail models have motivated a growing body of work on data-driven and hybrid modeling approaches. These methods utilize machine learning for5 acceleration of classical simulations, estimation of unmeasured internal quantities, or enabling of real-time decision support. In general, models maintain an interpretable physics-based backbone where possible, but this line is blurred when more advanced "black-box" models are introduced. Existing efforts, in general, fall into three overlapping categories: surrogate replacement of expensive physics submodels, data-driven estimation of in-train forces from operational data, and surrogate modeling for optimization and fleet/network level decision-making.

### 2.2.1 Surrogate Models for Accelerating Physics-Based Simulation

One prominent line of research focuses on embedding learned surrogate components within classical multibody or finite-element-based simulation frameworks. Instead of replacing the entire physics model with a "black-box" technique, these approaches substitute the most computationally intensive or nonlinear subcomponents with data-driven approximations trained on simulation data. For example, surrogate modules trained on finite-element outputs have been used to approximate nonlinear mechanical behavior within multibody vehicle–track simulations, showing a significant reduction in runtime while preserving acceptable accuracy [16]. Polynomial regression, Kriging, and other response-surface techniques are commonly employed in this setting, as they are highly interpretable and allow scientists and engineers to make grounded deductions.

More recent work has emphasized system integration of machine learning surrogates into digital twin architectures. In this schema, a full multibody dynamics (MBD) model is

replaced by a neural network based surrogate and packaged using standardized interfaces such as the Functional Mock-up Interface (FMI). This enables deployment as a Functional Mock-up Unit (FMU) within larger co-simulation or decision-support environments [17]. These studies demonstrate speed increases multiple orders of magnitude greater than classical MBD solvers. Furthermore, these studies demonstrate practical pipelines for continuous model deployment. However, they remain primarily focused on reproducing nominal simulator behavior rather than modeling uncertainty or difficult component interactions.

### 2.2.2 Data-Driven Estimation of In-Train Forces

A second major branch of data-driven rail modeling targets the estimation of in-train forces, which are difficult to measure directly in operational freight and passenger trains. Leveraging the increasing availability of onboard telemetry, particularly from Automatic Train Operation (ATO) systems, several authors have proposed supervised learning approaches that map operational signals such as speed, acceleration, and control commands to coupler force estimates. Convolutional neural networks (CNN) augmented with causal structure and self-attention mechanisms have demonstrated accurate reproduction of simulator-derived force histories under typical operating conditions [18].

Extensions of this concept incorporate multi-task learning and transfer learning to generalize force estimation across multiple couplers and train configurations. The In-trainNet framework, for example, combines shared feature extraction with configuration-specific adaptation to improve performance across different consists [19]. While these approaches represent an important step toward scalable "virtual instrumentation" of in-train forces, they are generally trained on labeled data generated by longitudinal train simulators and are validated on limited scenario sets.

### 2.2.3 Surrogate Modeling for Operations and Optimization

Data-driven surrogates have also been employed to accelerate optimization problems in railway operations, transcending the low-level dynamics and force estimation problems and solving high-level, or even fleet level problems. In the paper from Liu et. al., surrogate models are used to approximate the evaluation of candidate rail schedules, significantly reducing the amount of expensive computation required during optimization [20]. These methods often decompose the problem into multiple surrogate submodules and employ techniques such as transfer learning. Although effective in reducing computational burden, such surrogates are typically designed for specific operational contexts and rely on deterministic predictions of performance metrics for validation.

### 2.2.4 Limitations of Existing Data-Driven and Hybrid Approaches

Despite demonstrated success on several rail domains, existing data-driven and hybrid rail models exhibit several notable limitations. First, the majority of proposed surrogates are deterministic, providing point estimates without measures of predictive uncertainty. This limits their reliability when deployed outside the training distribution or when embedded

within control and optimization loops where constraint violations can have significant consequences.

Second, generalization across operating schemes and consist configurations remains a persistent challenge. Many force-estimation models are explicitly trained on a narrow subset of scenarios, often excluding braking modes, loading variations, or complex draft-gear hysteresis effects [18]. Even when transfer learning is employed, performance degradation is observed in domains where large strong nonlinear interactions between vehicles dominate [19]. These issues are exacerbated by the reliance on simulated labels, as discrepancies between simulation and real-world behavior can compound errors in purely data-driven mappings.

Finally, while hybrid surrogate approaches retain a physics-based structure, they are generally aimed at replication of nominal simulator outputs rather than modeling of epistemic uncertainty arising from unmodeled dynamics, parameter variability, or environmental disturbances. As a result, current surrogates offer limited robustness under distribution shift, constraining their applicability to safety-critical tasks such as energy-optimal control or real-time operational decision-making. Furthermore, the lack of baseline physical interpretability limits the industry adoption of new simulator technology, as rail engineers and scientists may be skeptical about a fully black-box prediction model (such as neural network).

### 2.2.5 Implications for Hybrid Probabilistic Modeling

Collectively, this body of work highlights the potential of data-driven methods to accelerate railway simulation and enable new monitoring capabilities, while also revealing fundamental gaps in uncertainty representation and out-of-distribution robustness. These limitations motivate hybrid modeling frameworks that explicitly combine physics-based foundational structure with probabilistic, data-driven components capable of representing uncertainty and adapting to previously unseen operating conditions.

## 2.3 Transformers for Dynamical Systems and Spatiotemporal Forecasting

Transformer architectures demonstrate strong candidacy for general purpose sequence modeling tasks. They have been increasingly adopted for dynamical systems modeling research and spatiotemporal forecasting. Transformers enable adaptive context aggregation over long temporal horizons by replacing recurrence with multi-head self attention. Unlike recurrent architectures, transformers do not impose a sequential dependency in their core computation. This intrinsic algorithmic feature allows transformers to be highly parallelizable during training. Transformers are particularly attractive for dynamical systems where future states often depend on long range temporal interactions, as trajectories can be represented as sequences of states or latent tokens [21].

### 2.3.1 Long-horizon Forecasting Transformers

A major branch of work in transformers is focused on long horizon temporal forecasting. Standard self-attention scales quadratically with sequence length, which severely limits it's

applicability to long-history prediction problems where fine-temporal resolution is necessary for accuracy. Informer addresses this problem by introducing ProbSparse attention, which identifies and attends to the most informative queries. This enables reduction in the computational cost for training in long horizon scenarios, but preserves enough necessary information to retain prediction accuracy. Compared to conventional models, Informer is capable of forecasting over substantially longer periods [22].

Autoformer is an additional architecture proposed by Wu et. al. which is motivated by the observation that many real-world time series phenomena (such as energy demand, traffic flow, and economic trends) are dominated by two distinct patterns. These include a slowly varying yet long-term trend, and repeating or periodic behavior (seasonal). At great computational cost, a neural network could be used to discover these structures implicitly. Autoformer builds these phenomena directly into the model, where the input time series is first decomposed into trend and seasonal components to enable the model to address long term evolution and short term noise/fluctuations separately. Further, Autoformer replaces scaled dot-product attention with an auto-correlation mechanism that explicitly searches for repeating temporal patterns across various time windows. If recurrent structures are identified, they can be reused more efficiently. Thus, Autoformer improves prediction accuracy and stability for long time series systems by reducing the computational burden of learning every single interaction between time steps [23].

Another promising architecture is the Temporal Fusion Transformer, which are designed for practical forecasting problems where predictions depend on many different types of information, not just a single time series data history. In applications, forecasting requires a combination of static attributes that do not change over time (such as system configuration or asset type), inputs that are known in advance (such as schedules, calendars, or planned control actions), and external variables that are observed over time but not controlled (such as weather or demand signals). Temporal fusion transformers partition these inputs instead of processing them all as a single, aggregate sequence. The model features mechanisms to determine which variables are the most relevant for prediction at each point in time. This structure allows TFT to generate multi-step forecasts while also providing interpretable information about which inputs most strongly influenced the predictions and when they were important, making the model more transparent and suitable for decision-support applications [24]. These properties make the model more transparent and interpretable, as compared to a black-box model.

### 2.3.2 Neural Operators for PDE-governed Dynamics

For spatiotemporal physical systems, forecasting can be cast as learning a solution operator mapping initial/boundary conditions (and coefficients/forcing) to future fields. Fourier Neural Operators (FNO) address forecasting and dynamical modeling problems from predicting individual timesteps to learning the governing function for how a system evolves. Instead of operating at specific values, FNO learns how information propagates across the entire spatial domain at once. As the name suggests, it does this by transforming the system state into the frequency domain, where long-term or broad patterns can be analyzed and computed much more efficiently. Within this new domain, the model applies global updates that resemble system-wide interactions rather than local point to point updates. As these learned updates

are defined independently of any spatial configuration, the resulting model can be evaluated on finer or coarser spatial grids than those seen during training. This allows FNO based models to generalize across different spatial resolutions, making them well suited as fast, flexible surrogates for complex dynamical simulation [25].

Physics-informed neural operators (PINO) expand on operator-learning models by explicitly incorporating known physical laws into the training process. In many engineering or scientific systems, the governing equations for how the system evolves over time are at least partially known. PINO leverages this knowledge by penalizing violations of the underlying physical formulation (the differential equations governing the system) during training, rather than relying solely on error measurements from data. In practice, the model may be trained using available simulation or measurement data at a relatively coarse spatial or temporal resolution, while simultaneously evaluating how well predictions satisfy the governing physical equations at a finer resolution. The resulting model is a generalizable forecaster that respects known physical constraints. The framework allows for the creation of models even when data is limited or expensive to obtain [26].

Beyond representations that rely only on Fourier representations, Cao investigates how attention mechanisms (which were originally created to model sequences) can be used to directly learn the rules of evolution for dynamical systems [27]. The study compares Fourier-based representations, which emphasize global frequency patterns, with Galerkin-style representations, which are rooted in classical numerical methods that approximate solutions using weighted basis functions. In analyzing how self-attention interacts with these different representations, the work shows that no single formulation is universally optimal. The choice depends on the structure of the underlying system and the types of interactions that dominate its dynamic evolution. This perspective reframes operator learning as an architectural design problem, where decisions about how to represent space, enforce structure, and aggregate information are as important as the learning algorithm itself (Cao, 2021).

More recently, attention-based models have been adapted specifically for learning the evolution of systems governed by partial differential equations. In this line of work, Transformer-style architectures are reformulated to learn solution operators that map initial conditions, system parameters, and external inputs directly to future system states. As opposed to advancing the state forward step by step in time, the model learns a global input–output relationship that captures how the entire state evolves. The attention mechanism enables the model to aggregate information across spatial locations and time, allowing it to represent long-range interactions that are common in physical systems. Utilizing Transformer architectures within an operator-learning framework and evaluating them on canonical PDE benchmarks, this work demonstrates that attention-based models can serve as effective and general surrogates for complex spatiotemporal dynamics, further validating the use of Transformer-style mechanisms in scientific and engineering modeling contexts [28].

### 2.3.3 Implications for Train Dynamics and Spatiotemporal Decision Support

These elements motivate hybrid modeling strategies for systems like train longitudinal dynamics where dynamics are structured (physics, constraints, conservation-like relationships)

yet exhibit long-range dependencies (consist interactions, route profile memory, control plans). Forecasting-oriented Transformers (Informer/Autoformer/TFT) supply scalable mechanisms for long-horizon sequence prediction and covariate fusion [22–24], while neural-operator perspectives (FNO/PINO and attention-based operator learners) suggest a principled route to learning dynamics as operators that map route/consist/control inputs to trajectories or distributions over outcomes [25–28]. These approaches collectively justify treating dynamical response prediction as (i) sequence-to-sequence forecasting with structured attention and (ii) operator learning with physics-aware regularization, providing a foundation for hybrid physics–Transformer surrogates in spatiotemporal control and planning.

## 2.4 Probabilistic Forecasting and Uncertainty Calibration

In engineered systems, point predictions alone are not sufficient for supporting reliable decision-making. In safety-critical domains, decisions must account for both the expected system behavior as well as the likelihood and severity of rare or adverse outcomes. In such environments, uncertainty is not a nuisance to be minimized, but a fundamental quantity that must be modeled, quantified, and communicated explicitly. Probabilistic forecasting provides a well-studied framework for this purpose by representing predictions as distributions rather than a single value output.

Probabilistic forecasts allow for estimation of quantities relevant to decision making such as prediction intervals, quantiles, and exceedance probabilities. These quantity estimations become directly actionable in downstream tasks such as risk-aware control, constraint satisfaction, and safety margins. For example, rather than predicting a single value representing a failure or violation, a probabilistic model can estimate the probability that a failure will happen.

A central concept of probabilistic forecasting is calibration, and refers to the statistical consistency between predicted probabilities and observed empirical frequencies. A probabilistic model is well-calibrated if events predicted to occur with probability $p$ indeed occur approximately $p$ fraction of the time in practice. Calibration is essential in safety-critical applications, where overconfident predictions can lead to gross underestimation of risk and potentially catastrophic decisions. Poor calibration undermines trust in probabilistic outputs, regardless of the apparent sharpness or sophistication of the predictive model.

However, good calibration alone is not sufficient for a strong probabilistic forecasting model. Trivially wide prediction intervals or overly diffuse distributions may be perfectly calibrated yet provide little useful information for decision-making. This motivates the complementary notion of sharpness, which measures the concentration or informativeness of probabilistic forecasts. Sharpness is a property of the predictive distribution itself, independent of the realized outcomes. Sharpness reflects the model's ability to make precise predictions. The fundamental tension between calibration and sharpness is well established in the forecasting literature: improving one often degrades the other. Evaluation of probabilistic models must therefore consider both aspects jointly rather than optimizing either in isolation.

In practice, many modern machine learning models produce uncertainty estimates with poor calibration, even when they achieve strong point prediction accuracy. This issue is exacerbated in deep learning models, which are often highly overconfident and sensitive to distributional shift. For safety-critical systems, this behavior is unacceptable, as it may systematically underestimate tail risk. Consequently, conservative uncertainty estimates are often preferred, even at the expense of reduced sharpness, provided that statistical guarantees can be established.

### 2.4.1 Foundations of Probabilistic Regression

Probabilistic regression aims to characterize the conditional distribution of a target variable given observed inputs, rather than producing a single point estimate. This perspective has a long history in statistics and has gained renewed importance in modern machine learning as uncertainty quantification has become central to decision-making in safety-critical and risk-sensitive applications. Existing approaches to probabilistic regression can be broadly categorized into quantile-based methods, Bayesian formulations, and likelihood-based probabilistic models, each of which encodes uncertainty in a fundamentally different manner.

Quantile regression, introduced by Koenker and Bassett [29], generalizes classical least-squares regression by estimating conditional quantiles of the response variable rather than its conditional mean. By minimizing an asymmetric absolute loss function, often referred to as the pinball or check loss, quantile regression provides a direct and distribution-free mechanism for estimating specific quantiles of the conditional outcome distribution.

This formulation has made quantile regression particularly attractive in applications where tail behavior and asymmetric risk are of interest. Unlike likelihood-based approaches, quantile regression does not require explicit assumptions about the parametric form of the underlying distribution, and quantiles can be estimated independently at arbitrary probability levels. However, classical quantile regression also exhibits important limitations. Independently estimated quantiles are not guaranteed to be mutually consistent, leading to the possibility of quantile crossing, and the method does not provide formal guarantees on coverage. As a result, estimated quantiles may systematically under- or over-cover the true conditional distribution, especially under model misspecification or distributional shift.

Bayesian neural networks represent a fundamentally different approach to probabilistic regression, in which uncertainty is modeled through probability distributions over model parameters rather than through direct estimation of output quantiles. In this framework, predictive uncertainty arises from marginalization over a posterior distribution on network weights. Variational inference methods have been proposed to make Bayesian neural networks tractable for large-scale models, most notably the Bayes by Backprop approach of Blundell et al. [30], which approximates the posterior distribution over weights using parametric variational families.

Approximate Bayesian techniques such as Monte Carlo dropout further reduce computational complexity by interpreting dropout at inference time as a variational approximation to Bayesian inference [31]. These methods have been widely adopted due to their ease of implementation and compatibility with standard deep learning architectures. Bayesian neural networks and their approximations are often interpreted as capturing epistemic uncertainty arising from limited data or model uncertainty.

Despite their conceptual appeal, Bayesian neural networks do not inherently guarantee calibrated predictive distributions. The quality of uncertainty estimates depends strongly on the choice of prior, the expressiveness of the variational approximation, and the optimization procedure. In practice, approximate inference error and overparameterization can lead to miscalibrated predictions, particularly in regions of the input space that are sparsely represented in the training data.

In addition to quantile-based and Bayesian approaches, many modern regression models produce probabilistic outputs by directly parameterizing a predictive likelihood. Common formulations assume a parametric distribution, such as a Gaussian with learned mean and variance, and train the model using maximum likelihood objectives. More expressive variants, including mixture density networks, extend this approach by modeling multimodal conditional distributions.

Likelihood-based probabilistic regressors offer strong expressive power and are straightforward to integrate into deep learning pipelines. However, their probabilistic validity depends critically on the correctness of the assumed likelihood model. When model assumptions are violated, learned uncertainty estimates may be systematically biased, resulting in overconfident or underconfident predictions. Empirical evidence suggests that such models can achieve high point prediction accuracy while producing poorly calibrated uncertainty estimates, limiting their reliability in downstream decision-making.

The existing literature on probabilistic regression provides a diverse set of tools for uncertainty-aware prediction. Quantile regression offers a simple and distribution-free approach to estimating conditional quantiles but lacks formal coverage guarantees. Bayesian neural networks provide a principled framework for modeling epistemic uncertainty but are sensitive to approximation quality and modeling assumptions. Likelihood-based probabilistic regressors are flexible and scalable but often exhibit miscalibrated uncertainty in practice.

These limitations motivate subsequent work on uncertainty calibration and distribution-free guarantees. In particular, methods that explicitly evaluate and correct calibration, as well as techniques that provide finite-sample coverage guarantees independent of model correctness, have emerged as important complements to expressive probabilistic regression models.

### 2.4.2 Empirical Miscalibration in Deep Models

Despite their expressive power and strong performance in point prediction tasks, modern deep learning models frequently produce probabilistic predictions with poor calibration. This phenomenon has been systematically documented by Guo et al. [32], as they demonstrated that contemporary neural networks tend to produce overconfident results, even when trained with likelihood-based objectives and regularization techniques. In both classification and regression settings, predicted probabilities often fail to correspond to empirical frequencies. This damages trust in the model to produce reliable estimates in downstream decision-making.

A major contribution of this thread of work is the formalization and adoption of calibration diagnostics for neural networks. Metrics such as expected calibration error (ECE) and reliability diagrams provide both quantitative and visual tools to analyze whether predicted confidence levels align with observed outcomes. Empirical results indicate that increasing

model capacity, depth, and training duration can contribute to miscalibration, despite improvements in predictive accuracy. These findings challenge the assumption that probabilistic outputs produced by deep models are inherently reliable.

Miscalibration issues are not confined to only deterministic neural networks. Bayesian neural networks and approximate Bayesian methods (such as variational inference and Monte Carlo dropout) have also been shown to produce miscalibrated predictive distributions. Approximation error in posterior inference, sensitivity to prior specification, and artifacts of optimization can all contribute to overconfident uncertainty estimates. As a result, Bayesian formulations alone do not guarantee calibrated predictions, particularly in high-dimensional or data-sparse schemes.

The presence of miscalibration in deep probabilistic models has significant implications for safety-critical applications. Overconfident predictions can lead to systematic underestimation of risk, while underconfident predictions may result in overly conservative behavior and reduced efficiency. These observations motivate the need for uncertainty quantification methods that go beyond expressive modeling and explicitly address calibration, evaluation, and robustness. Subsequent work has therefore focused on calibration-aware learning objectives, distribution-free guarantees, and evaluation frameworks that jointly consider calibration and sharpness, which are reviewed in the following sections.

### 2.4.3 Post-hoc Calibration for Regression

The empirical miscalibration observed in modern probabilistic models (especially deep models) has motivated the development of post-hoc calibration methods that operate independently of the underlying predictive model. Kuleshov et al. [33] formalized calibration for regression by defining a notion of distributional consistency based on predictive cumulative distribution functions. Under this framework, a probabilistic regressor is calibrated if the empirical frequency of outcomes falling below predicted quantiles matches the nominal quantile levels.

Expanding on this definition, Kuleshov et al. proposed a general recalibration procedure that maps predicted cumulative probabilities to calibrated probabilities using held-out validation data. This approach is model-agnostic and can be applied to a wide range of probabilistic regressors, including Bayesian neural networks and likelihood-based deep models. Empirical results demonstrate substantial improvements in calibration with minimal degradation in predictive sharpness, making calibrated regression a practical and widely applicable correction mechanism.

However, post-hoc calibration methods rely on the assumption that calibration error is stable across training and deployment distributions. While effective in correcting average miscalibration on validation data, these techniques do not provide formal guarantees on predictive coverage and may fail under distributional shift or in sparsely sampled regions of the input space. As a result, post-hoc calibration improves empirical reliability but does not eliminate the need for uncertainty estimation methods with stronger statistical guarantees.

Figure 2.1 illustrates the distinction. A well-calibrated model produces quantiles such that when we predict the $q$-quantile $\hat{Q}_q(X)$, the fraction of held-out outcomes $Y \leq \hat{Q}_q(X)$ is approximately $q$. The diagonal represents this ideal. An uncalibrated model may deviate: for example, an overconfident model (underestimated variance) yields empirical coverage

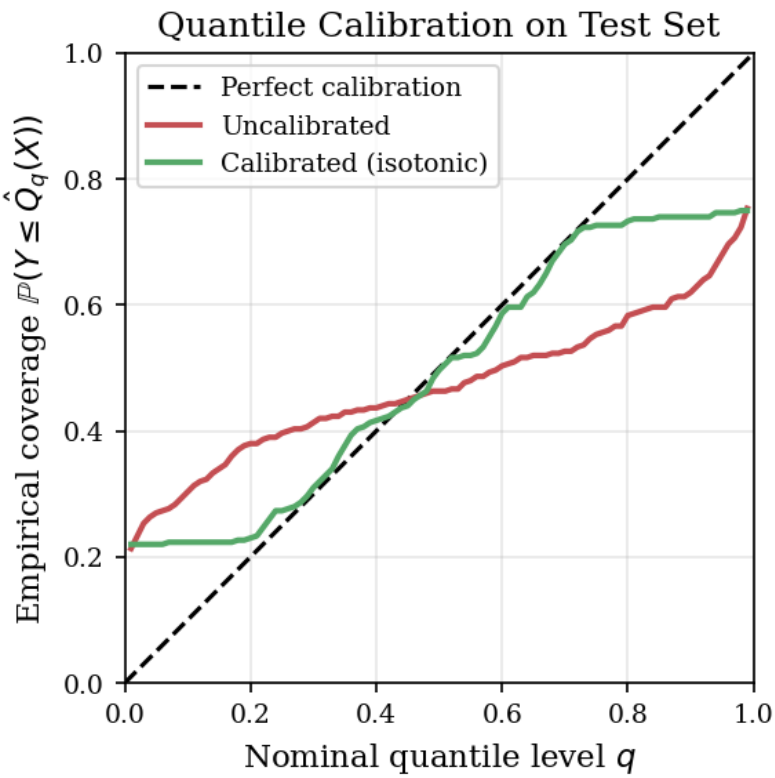


Figure 2.1: Quantile calibration: diagonal = perfect; uncalibrated model deviates; isotonic calibration corrects.

below the diagonal at high nominal levels. Post-hoc calibration methods such as isotonic regression can correct this by learning a monotone mapping from predicted to calibrated quantile levels on a validation set.

### 2.4.4 Distribution-Free Coverage via Conformal Prediction

Conformal prediction offers an alternative approach to uncertainty quantification by providing finite-sample, distribution-free coverage guarantees under minimal assumptions. Introduced by Shafer and Vovk [34], conformal prediction constructs prediction sets that contain the true outcome with a specified probability, provided the data are exchangeable. Crucially, these guarantees hold regardless of the correctness of the underlying predictive model.

Recent work has integrated conformal prediction with quantile regression to produce calibrated prediction intervals that adapt to heteroscedasticity in the data. Romano et al. [35] proposed conformalized quantile regression, which adjusts estimated conditional quantiles to ensure exact marginal coverage. This method preserves the flexibility of quantile regression while enforcing rigorous statistical guarantees, making it particularly attractive for safety-critical applications.

The primary limitation of conformal prediction methods is their inherent conservatism. Coverage guarantees are typically marginal rather than conditional, and the resulting prediction intervals may be wider than necessary for individual inputs. This trade-off between

coverage guarantees and predictive sharpness reflects a fundamental tension in uncertainty quantification. Regardless, conformal prediction provides a robust and model-agnostic mechanism for uncertainty estimation, complementing calibration-based approaches and motivating hybrid methods that balance expressiveness, calibration, and statistical rigor.

### 2.4.5 Proper Scoring Rules and Evaluation Metrics

Evaluating probabilistic forecasts requires metrics that jointly assess both calibration and sharpness. Proper scoring rules provide a principled framework for this purpose by assigning a numerical score to a predicted distribution such that the expected score is minimized when the predictive distribution matches the true data-generating distribution. Gneiting and Raftery [36] formalized this framework and emphasized that calibration and sharpness must be considered together rather than as competing objectives.

Commonly used proper scoring rules include the logarithmic score, the continuous ranked probability score (CRPS), and interval-based scores. Unlike point-error metrics, proper scoring rules penalize both overconfident and overly diffuse predictions, making them well suited for comparing probabilistic models. As a result, they have become standard tools for evaluating probabilistic forecasts across a wide range of applications, including time series prediction and uncertainty-aware regression.

In the context of safety-critical forecasting, proper scoring rules provide an objective means of assessing whether probabilistic predictions are both reliable and informative. Their use complements calibration diagnostics and coverage-based guarantees, offering a unified evaluation framework for probabilistic regression models.

## Synthesis and Implications

Taken together, the literature reviewed in this chapter highlights both the maturity and the limitations of existing approaches to train dynamics modeling and decision support. Classical longitudinal dynamics models provide well-understood physical structure and interpretability but remain computationally expensive and difficult to integrate into optimization and control pipelines. Data-driven and hybrid modeling efforts have demonstrated the feasibility of surrogate models for accelerating simulation, estimating in-train forces, and supporting operational decision-making, yet these approaches are typically deterministic, limited in their ability to generalize outside the training distribution, and rarely equipped with rigorous uncertainty quantification. Recent advances in transformer architectures for dynamical systems and spatiotemporal forecasting offer powerful tools for modeling long-horizon dependencies and high-dimensional temporal structure, but their application to rail systems remains largely unexplored and their uncertainty estimates are often informal or empirically calibrated at best. Finally, the probabilistic forecasting literature establishes that expressive models alone do not yield reliable uncertainty estimates, motivating the use of calibration techniques, distribution-free coverage guarantees, and proper scoring rules to ensure statistical reliability in safety-critical settings. Collectively, these findings suggest a gap between physically grounded simulation, expressive data-driven models, and trustworthy probabilistic outputs. This gap motivates the development of hybrid modeling frameworks that combine

physics-based structure, transformer-style sequence modeling, and explicitly calibrated probabilistic forecasting to support risk-aware train dynamics simulation and decision-making.

# Chapter 3

# Problem Formulation

## 3.1 Forecasting problem, classical dynamics, and tensorized formulation

This chapter specifies the prediction problem addressed in the thesis and the mathematical objects on which it rests. The physical model is classical longitudinal train dynamics: a chain of coupled vehicles moving along a route, with traction, braking, resistance, grade, and coupler mechanics entering a force balance for each vehicle. What is distinctive in the present work is not a claim that the rail-dynamics literature is historically cast in tensor form, but that *we recast* the same coupled-vehicle system into a structured tensor representation so that simulation outputs, learning targets, and sequence-based surrogate models share a common layout. The following subsections state the operational prediction task, summarize the classical equations, introduce the node and edge tensors used throughout later chapters, and connect them to the forecasting and operator-learning viewpoint. The detailed stage-by-stage reference simulator is then documented in section 3.2; readers may consult it for sign conventions, nonlinear coupler laws, and the extended actuator model.

### 3.1.1 Physical setting and prediction task

Consider a freight train moving longitudinally along a prescribed path. Operational planning and safety assessment require predicting how the consist responds to the route, its makeup, and the control actions applied over a horizon. Concretely, one specifies a *route profile* (geometry and constraints along track), a *consist configuration* (ordered vehicles, masses, resistance parameters, coupler data), and a *control or operating plan* (traction and braking commands, possibly in feedback with a driver or automation model). Given these inputs, the quantities of interest include, among others:

- **Longitudinal motion:** positions and velocities of each vehicle, and derived accelerations where needed.

- **In-train forces:** coupler forces and related deformation measures between adjacent vehicles, which govern draft and buff loading.

- **Energy and propulsion:** work, power, or fuel-related summaries integrated along the run.
- **Schedule and performance:** trip time, arrival time at milestones, or delay relative to a target.
- **Risk and margins:** exceedance of force or speed limits, or other constraint margins, especially when predictions are probabilistic.

The thesis treats the simulator that maps these inputs to trajectories and summaries as a deterministic *reference* when parameters are fixed, and augments that view with uncertainty in parameters and environment when forecasting is required to be uncertainty-aware (section 3.5).

### 3.1.2 Classical longitudinal dynamics formulation

Classical longitudinal modeling represents the train as a chain of $N$ vehicles indexed $i = 0, \ldots, N-1$ from lead to tail, connected by $N-1$ couplers. Let $x_i(t)$ and $v_i(t) = \dot{x}_i$ denote the longitudinal position and velocity of vehicle $i$ along the route, and let $m_i$ denote its mass. Between vehicles $i$ and $i+1$, coupler $i$ is associated with a kinematic extension $\delta_i$ (relative displacement from nominal spacing, positive in draft in the convention used below) and a coupler force $F_i$ transmitted to the trailing vehicle in the pair. Resistance, grade, and optional curvature terms enter as external longitudinal forces on each vehicle; traction and braking appear as controlled inputs, possibly after actuator dynamics. Summarizing the external and control contributions on vehicle $i$ by $F_{\text{ext},i}(t)$ and the coupler incidence by the force on the rear of $i$ minus the force on the front, the translational dynamics take the familiar form

$$\dot{x}_i = v_i, \qquad m_i \dot{v}_i = F_{\text{ext},i}(t) + F_{i-1}(t) - F_i(t), \tag{3.1}$$

with $F_{-1} = F_{N-1} = 0$ when end forces are absent. The coupler forces $F_i$ are determined by constitutive laws $F_i = \phi_i(\delta_i, \dot{\delta}_i, \ldots)$ linking kinematics to internal forces; slack, asymmetric buff and draft stiffness, and damping enter here (section 3.2). When actuator lag or power limits are modeled, eq. (3.1) is augmented by additional state variables per vehicle (for example brake and traction command states) driven by exogenous commands. This stacked system is the physical foundation: it is standard in multi-body longitudinal modeling and does not depend on the tensor notation introduced next.

### 3.1.3 Tensorized state representation

For learning and for software that must handle variable train length and rich per-vehicle features, it is convenient to organize the same physics into two matrices that make the vehicle–coupler graph explicit. In this thesis, a *node* (vehicle) tensor $\mathbf{H}(t) \in \mathbb{R}^{N \times d_n}$ collects $d_n$ channels associated with each vehicle at time $t$. Typical channels include longitudinal position and velocity, internal actuator or command states when those are part of the dynamic model, and static attributes replicated on each row (mass, Davis coefficients, traction capability flags, force limits, and similar parameters) so that a single array suffices for batched

computation and for graph- or attention-style models that read local vehicle features. An *edge* (coupler) tensor $\mathbf{E}(t) \in \mathbb{R}^{(N-1)\times d_e}$ holds $d_e$ channels per coupler, such as extension $\delta_i$, relative speed $\dot{\delta}_i$, algebraic coupler force $F_i$, nominal spacing, slack half-width, and stiffness and damping parameters for buff and draft segments. The assignment of channels is fixed in the reference implementation; the important point is the separation between vehicle-centered quantities in $\mathbf{H}$ and coupler-centered quantities in $\mathbf{E}$.

Along a discrete time grid $t_0, \ldots, t_{T-1}$, simulated trajectories can be stored as a *rollout tensor* $\mathbf{X} \in \mathbb{R}^{T\times N\times d}$, whose time slice $\mathbf{X}_{k,:,:}$ is the node feature matrix at $t_k$ (so one may take $d = d_n$ and identify $\mathbf{X}_{k,:,:}$ with $\mathbf{H}(t_k)$ after any bookkeeping for channels). Coupler histories are stored analogously as $\mathbf{E}_{\text{hist}} \in \mathbb{R}^{T\times(N-1)\times d_e}$. This layout preserves the sparsity and locality of longitudinal coupling: each coupler row depends only on two adjacent vehicle rows, matching the chain topology of eq. (3.1). It also supports variable $N$ by padding and masking in learning pipelines, and it aligns naturally with transformer and other sequence models that expect token sequences with consistent feature dimension.

### 3.1.4 Dynamics in tensor-oriented form

The tensor view is a reorganization of state and parameters, not a change to the underlying force balances. Let $\mathbf{H}_{\text{dyn}}(t)$ denote the dynamic channels within $\mathbf{H}(t)$ (for example positions, velocities, and actuator states in the extended reference model), and let $\mathbf{p}$ collect fixed parameters, including static columns of $\mathbf{H}$ and of $\mathbf{E}$ where those encode coupler constants. The differential part of the evolution can be written compactly as

$$\frac{\mathrm{d}}{\mathrm{d}t}\,\mathrm{vec}(\mathbf{H}_{\text{dyn}}(t)) = \mathcal{F}_n\big(\mathbf{H}(t), \mathbf{E}(t), \mathcal{U}(t), \mathcal{R}; t\big), \tag{3.2}$$

where $\mathcal{U}$ denotes the control input signals, $\mathcal{R}$ the route profile, and $\mathcal{F}_n$ the assembled right-hand side obtained from eq. (3.1) and the actuator relations. Coupler kinematics link $\mathbf{H}_{\text{dyn}}$ to the extension and rate channels in $\mathbf{E}(t)$; constitutive laws determine the force channels. In the reference integrator used for data generation in this thesis, several edge channels (including $\delta_i$, $\dot{\delta}_i$, and $F_i$) are *reconstructed algebraically* at each evaluation from vehicle positions and velocities and from coupler parameters, rather than integrated as separate states. Thus the edge tensor is in part an explicit bookkeeping of quantities that satisfy relations of the form

$$\mathbf{E}(t) = \mathcal{G}_e\big(\mathbf{H}(t), \mathbf{p}\big), \tag{3.3}$$

for the algebraic channels, while eq. (3.2) carries the true differential degrees of freedom. If additional internal coupler or draft-gear states were introduced in future work, eq. (3.3) would be supplemented by differential equations for the corresponding rows of $\mathbf{E}(t)$, i.e. a nontrivial $\mathcal{F}_e$ on those components alongside $\mathcal{F}_n$. The present wording is general enough to cover both cases without overstating what the current code integrates.

### 3.1.5 Forecasting objective and simulator operator

The learning problem is to predict, under computation and uncertainty constraints, the outputs described in section 3.1.1 from the structured inputs $(\mathcal{R}, \mathcal{C}, \mathcal{U})$. The reference physics

implements a deterministic operator

$$F : (\mathcal{R}, \mathcal{C}, \mathcal{U}) \mapsto \mathbf{y}, \tag{3.4}$$

where $\mathbf{y}$ collects trajectories (for example stacks of $\mathbf{H}(t_k)$ and $\mathbf{E}(t_k)$), scalar summaries such as energy and trip time, and any derived risk indicators. The triple $(\mathcal{R}, \mathcal{C}, \mathcal{U})$ denotes the same structured inputs as the route profile $\mathcal{R}$, consist $\mathcal{C}$, and control plan $\mathcal{U}$ detailed in section 3.3. When uncertain parameters and environment are modeled by a random vector $\xi \sim p(\xi)$, the same operator is written $F(\mathcal{R}, \mathcal{C}, \mathcal{U}; \xi)$ and outputs become random (section 3.5). The goal pursued in this thesis is not only pointwise trajectory accuracy but, where relevant, *calibrated* probabilistic forecasts over operationally meaningful quantities (forces, times, margins), so that exceedance statements and intervals can be trusted at stated levels. The surrogate architecture introduced later approximates $F$ (and its predictive distributions) using a hybrid physics-aware, sequence-based model whose inputs and outputs are aligned with the tensor layout above.

## 3.2 Reference longitudinal train dynamics model

The preceding section fixed the prediction problem and the tensorized packaging of state used in this work; here we summarize a transparent *reference* longitudinal train dynamics (LTD) model that was implemented incrementally for pedagogy and validation. The development follows a Jupyter notebook that constructs the simulator stage by stage using only NumPy, SciPy's `solve_ivp`, and Matplotlib: each stage adds a small number of physical effects while preserving a clear force-balance and sign convention. The same model is mirrored in a structured Python package that exposes node and edge tensors alongside the classical flat state for integration. Parameters (masses, Davis coefficients, coupler stiffness, and traction limits) are illustrative rather than calibrated to a particular operator or vehicle test programme; the purpose is to obtain a reproducible intermediate-fidelity integrator whose structure matches the multi-body, route-coupled problem stated above. Time integration uses adaptive Runge–Kutta or implicit methods as needed when stiff coupler dynamics appear. The figures in this section are exported directly from that notebook at representative numerical settings.

### 3.2.1 Stage 1: single powered vehicle

The starting point is a single point mass $m$ with longitudinal position $x(t)$ and velocity $v(t) = \dot{x}$. The equation of motion aggregates traction, braking, Davis running resistance, grade, and an optional curvature proxy into a single scalar force balance:

$$m\dot{v} = F_{\text{trac}}(t) - F_{\text{brk}}(t) - R(v) - G(x) - C(x, v), \tag{3.5}$$

with $\dot{x} = v$. Davis resistance is implemented in the usual magnitude–direction form,

$$R(v) = \left(A + B|v| + Cv^2\right) \operatorname{sgn}(v), \tag{3.6}$$

with $\operatorname{sgn}(v)$ set to zero at $|v| < \varepsilon$ to avoid chatter at standstill. Grade enters through $G(x) = mg \sin\theta(x)$ with $\theta$ the local track inclination. The optional term $C$ is a simple

longitudinal curvature proxy proportional to $mv^2|\kappa(x)|$ with tunable scale, disabled in the earliest runs. Traction and brake commands may be specified as arbitrary functions of time; the first stage uses a smooth "ramp" in time so that $F_{\text{trac}}$ rises continuously from zero after a delay, rather than stepping instantaneously, which eases numerical stiffness and mirrors the later actuator-lag extension. Figure 3.1 shows position, speed, and the plotted residual $F_{\text{trac}} - R(v)$ on level track with braking and curvature removed, confirming that the integrator and force signs behave as intended before couplers are introduced.

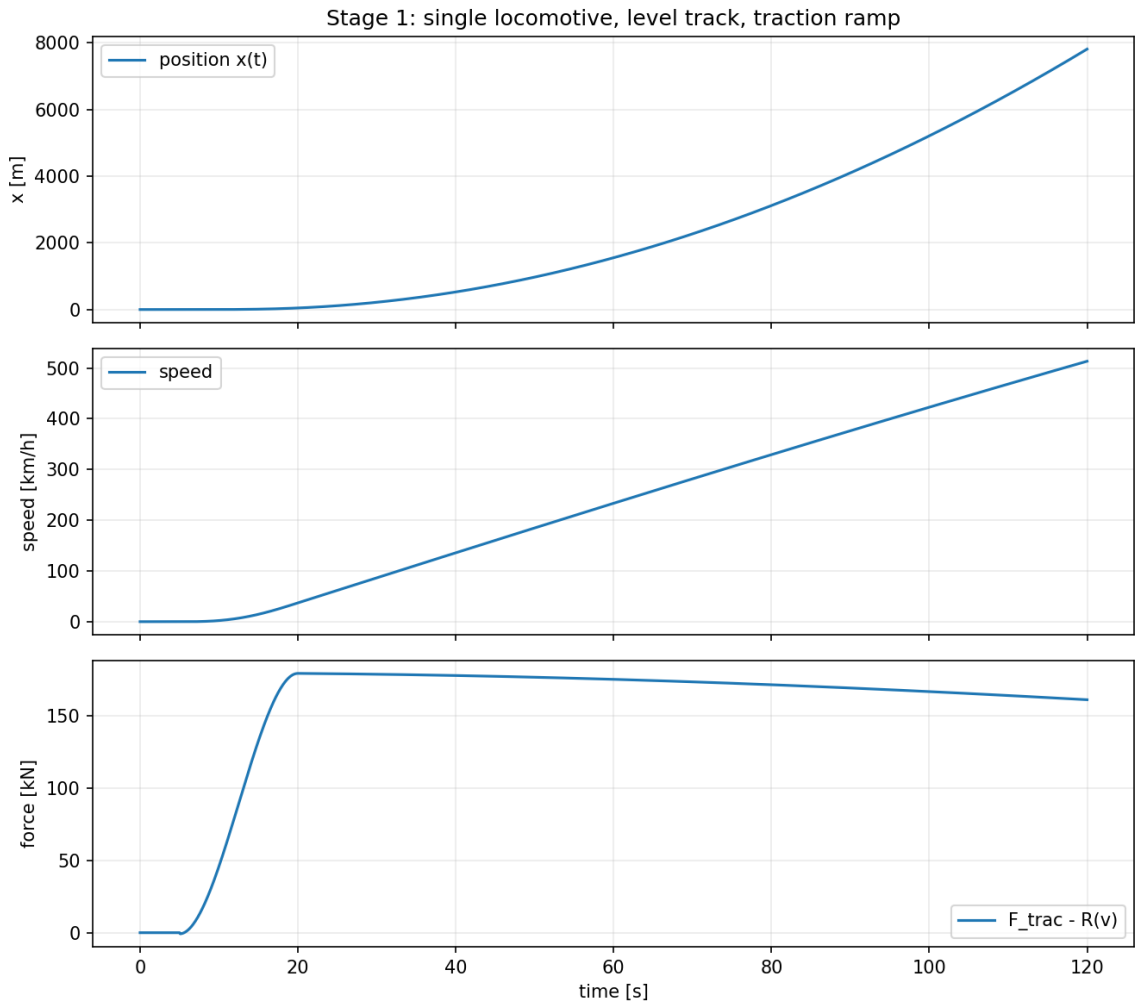


Figure 3.1: Stage 1 reference run: single vehicle on level track with a smooth traction ramp. Top to bottom: position, speed, and net longitudinal force $F_{\text{trac}} - R(v)$ versus time.

### 3.2.2 Stage 2: locomotive and one car with a linear coupler

A second mass $m_1$ is attached behind the lead vehicle $m_0$. Nominal centre-to-centre spacing is $L_0$. Coupler extension (positive in draft) is

$$\delta = (x_0 - x_1) - L_0, \qquad \dot{\delta} = v_0 - v_1. \tag{3.7}$$

A linear spring–damper coupler exerts force $F = k\delta + c\dot{\delta}$ on the rear car along the direction of travel; the lead feels $-F$. With $G_i$ and $R_i$ evaluated per vehicle,

$$m_0\dot{v}_0 = F_{\text{trac}} - F_{\text{brk},0} - R_0(v_0) - G_0(x_0) - F, \tag{3.8}$$

$$m_1\dot{v}_1 = -F_{\text{brk},1} - R_1(v_1) - G_1(x_1) + F. \tag{3.9}$$

The state vector is $(x_0, x_1, v_0, v_1)$. For stiff $(k, c)$ the two speeds track almost identically on a speed–time plot even though the coupler force and extension carry the load path; fig. 3.2 shows the characteristic transient in $F$ and $\delta$ as slack is taken up and the train settles into quasi-uniform acceleration under traction.

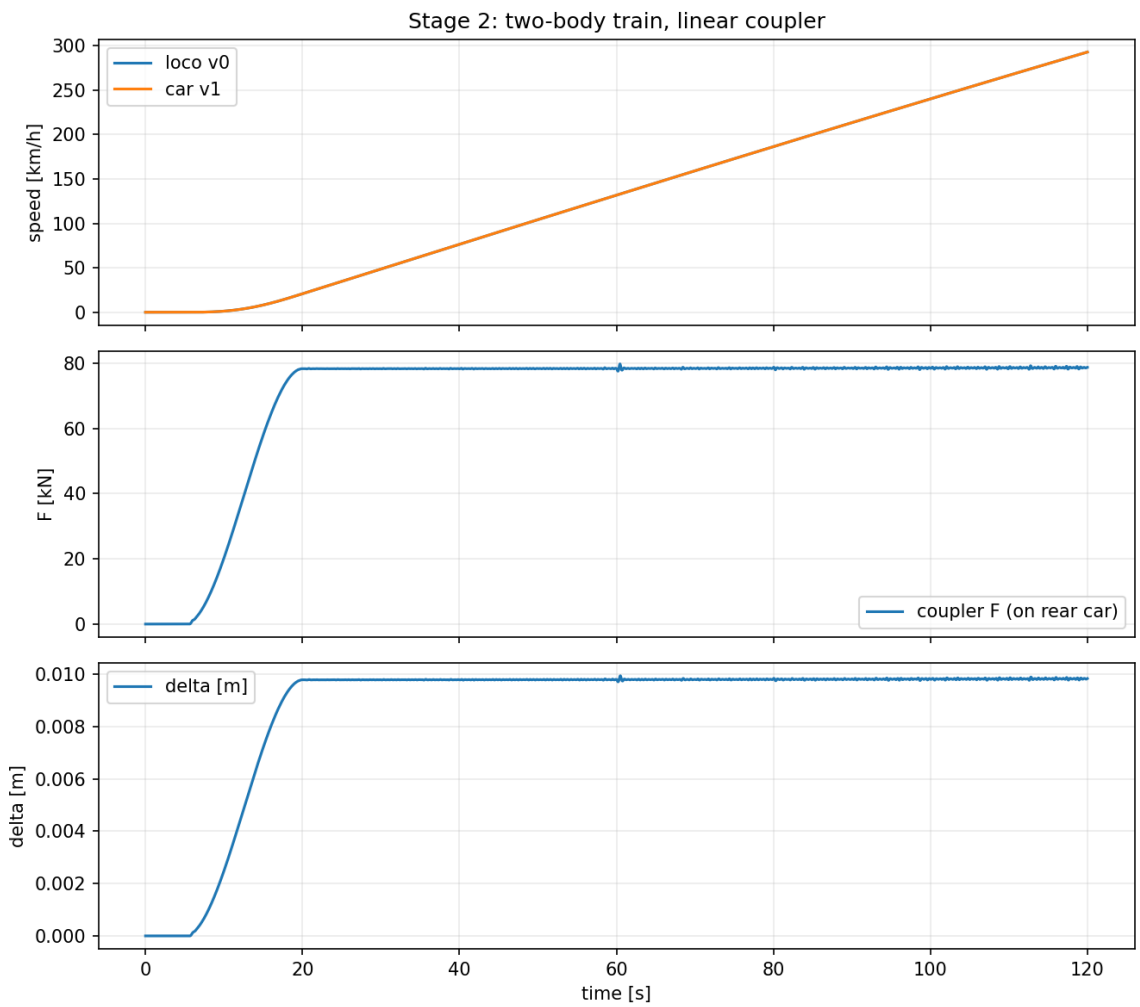


Figure 3.2: Stage 2: two-body train with linear coupler. Speeds of lead and trailing vehicle (overlapping at this scale), coupler force on the rear car, and extension $\delta$.

### 3.2.3 Stage 3: nonlinear coupler with slack and asymmetric buff/draft

The linear model is retained for comparison, but the simulated dynamics replace $F$ by a piecewise law with slack half-width $s$. For $|\delta| \leq s$ the coupler force is zero; for $\delta > s$

(draft) stiffness and damping $(k_\mathrm{d}, c_\mathrm{d})$ apply to the over-travel $\delta - s$, and for $\delta < -s$ (buff) a possibly different pair $(k_\mathrm{b}, c_\mathrm{b})$ applies to $\delta + s$. The same two-body equations eq. (3.9) hold with this $F(\delta, \dot{\delta})$. Figure 3.3 overlays the *linear* force $k\delta + c\dot{\delta}$ that would correspond to Stage 2 parameters against the *actual* nonlinear force along the trajectory, together with $\delta$ and $v_0 - v_1$. The deadband produces intervals of near-zero force and sharper loading when contact engages, which motivates slack-aware modelling for impact-style longitudinal loads.

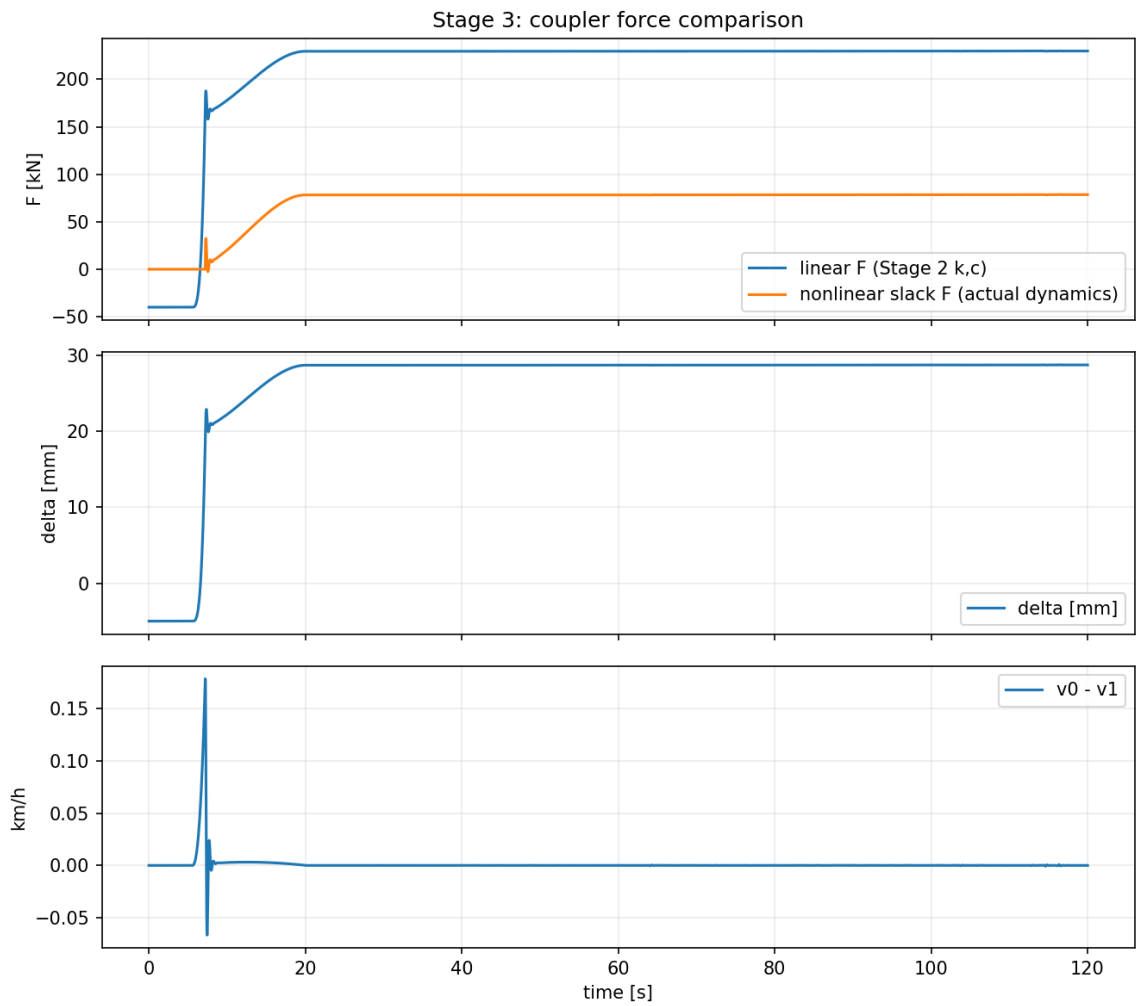


Figure 3.3: Stage 3: comparison of linear spring–damper force (Stage 2 parameters) versus nonlinear slack coupler force along the same trajectory, with extension and relative speed.

### 3.2.4 Stage 4: three bodies and sequential coupler forces

With vehicles indexed $0, 1, 2$ front to rear, coupler $j \in \{0, 1\}$ connects $j$ to $j + 1$ and $F_j$ denotes the force on the *rear* vehicle in that pair. The recursions generalise naturally:

$$m_0 \dot{v}_0 = F_{\text{trac}} - R_0 - G_0 - F_0, \tag{3.10}$$
$$m_1 \dot{v}_1 = -R_1 - G_1 + F_0 - F_1, \tag{3.11}$$
$$m_2 \dot{v}_2 = -R_2 - G_2 + F_1, \tag{3.12}$$

with each $F_j$ obtained from the nonlinear coupler law applied to $(\delta_j, \dot{\delta}_j)$ where $\delta_j = (x_j - x_{j+1}) - L_{0,j}$. End-to-end stretch relative to nominal spacing, $(x_0 - x_2) - (L_{0,0} + L_{0,1})$, summarises how the consist elongates in draft. Figure 3.4 shows speeds, both coupler forces, and stretch for a traction-only run. An optional variant applies a time-localised service-brake force on the lead only while traction continues, illustrating redistribution of draft forces (fig. 3.5).

### 3.2.5 Stage 5: general $N$-vehicle assembly

The state is stacked as $\mathbf{y} = (x_0, \ldots, x_{N-1}, v_0, \ldots, v_{N-1})^\top \in \mathbb{R}^{2N}$. For each coupler $j = 0, \ldots, N-2$, $\delta_j$ and $\dot{\delta}_j$ are defined as in eq. (3.7) with vehicle $j$ ahead of $j + 1$. The force $F_j$ on the rear vehicle follows the slack model with per-coupler parameters. The right-hand side for vehicle $i$ is

$$m_i \dot{v}_i = F_{\text{trac},i} - F_{\text{brk},i} - R_i(v_i) - G_i(x_i) - C_i + F_{i-1}^{\text{in}} - F_i^{\text{out}}, \tag{3.13}$$

with the convention $F_{-1}^{\text{in}} = F_{N-1}^{\text{out}} = 0$, and with traction and braking possibly zero on unpowered cars. Data structures (vehicle and coupler parameter records, and a flat route profile) group masses, Davis triples, coupler geometry, and slack/stiffness data for reuse. Figure 3.6 plots selected speeds and coupler forces for a consist with one powered lead and identical trailing cars; fig. 3.7 shows a space–time style heatmap of all coupler forces, illustrating the monotonic decay of steady draft force from head to tail when only the lead provides traction.

### 3.2.6 Stage 6: per-vehicle grade and curvature from a route profile

Grade and curvature are no longer evaluated at a single global $x$; each vehicle samples $\sin\theta(x_i)$ and optionally $\kappa(x_i)$ at its own position. The profile is stored as piecewise-linear nodes and interpolated (e.g. linearly in arc length). Then $G_i = m_i g \sin\theta(x_i)$ and the same curvature proxy as before applies per vehicle. This introduces spatial dispersion along the train: the head and tail can experience different grades simultaneously, which excites relative motion and coupler cycling even when speed traces remain nearly overlaid. Figure 3.8 shows the prescribed $\sin\theta$ along route distance in the top panel and the time evolution of lead and rear speed and selected coupler forces below; the horizontal axes of the lower panels are simulation time, not chainage, so the spatial profile should be read as a fixed map through which the train moves.

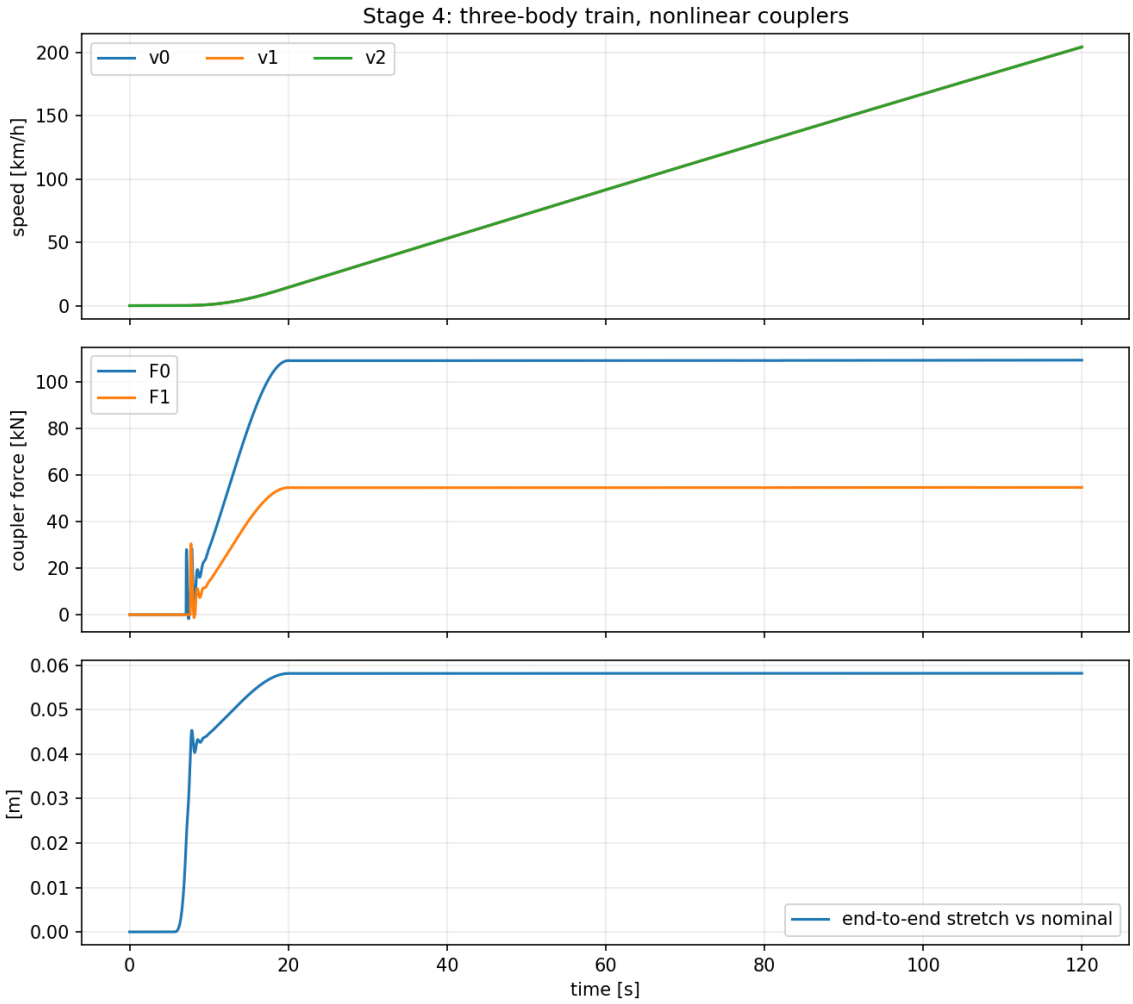


Figure 3.4: Stage 4: three-body train with nonlinear couplers. Speeds, coupler forces $F_0$ and $F_1$, and end-to-end stretch versus time.

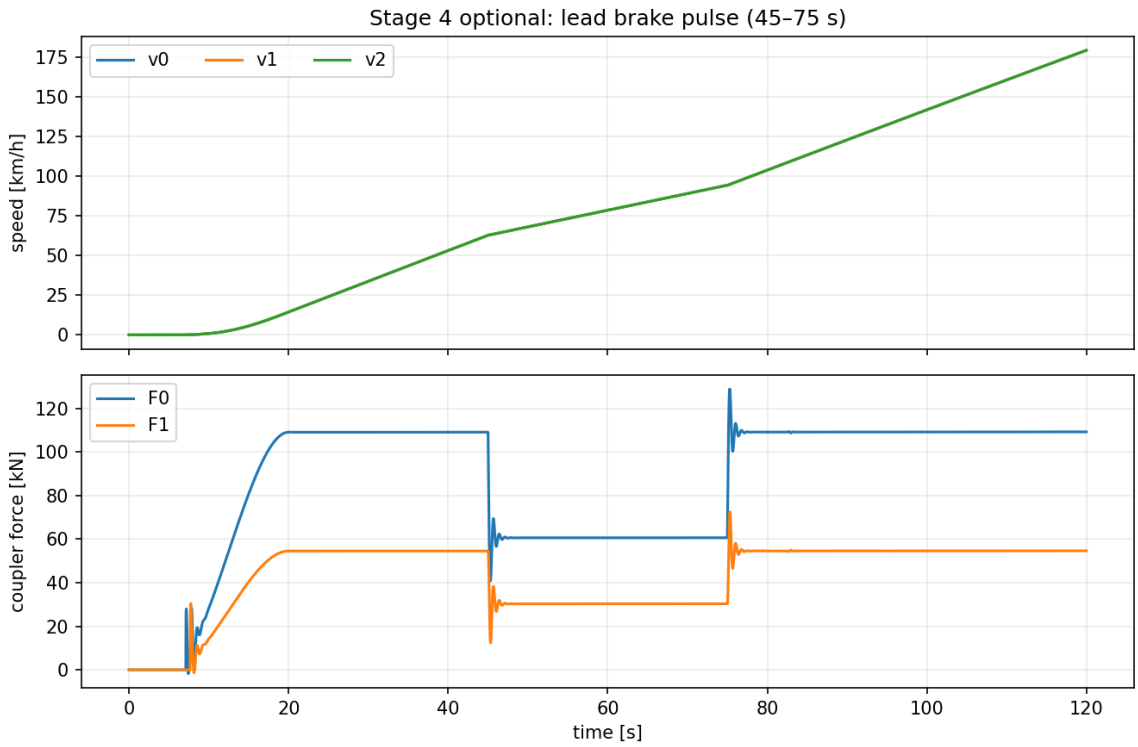


Figure 3.5: Stage 4 optional scenario: lead-only brake pulse; speeds and coupler forces.

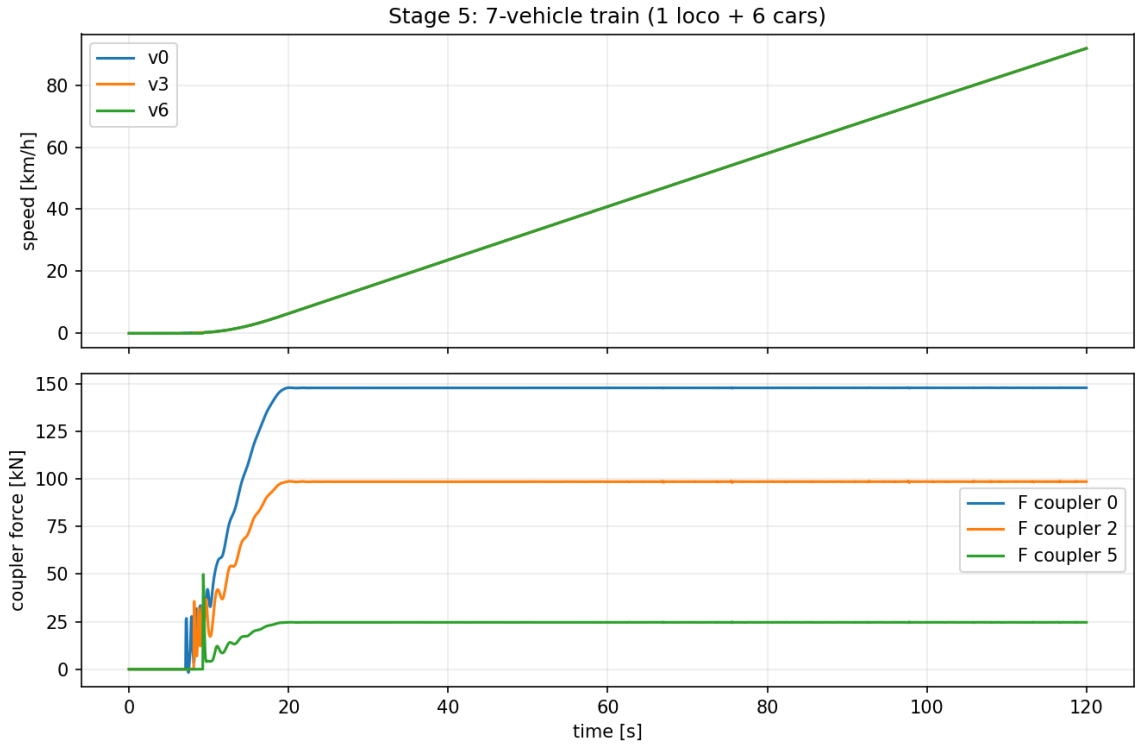


Figure 3.6: Stage 5: $N$-vehicle train; selected speeds and coupler forces versus time.

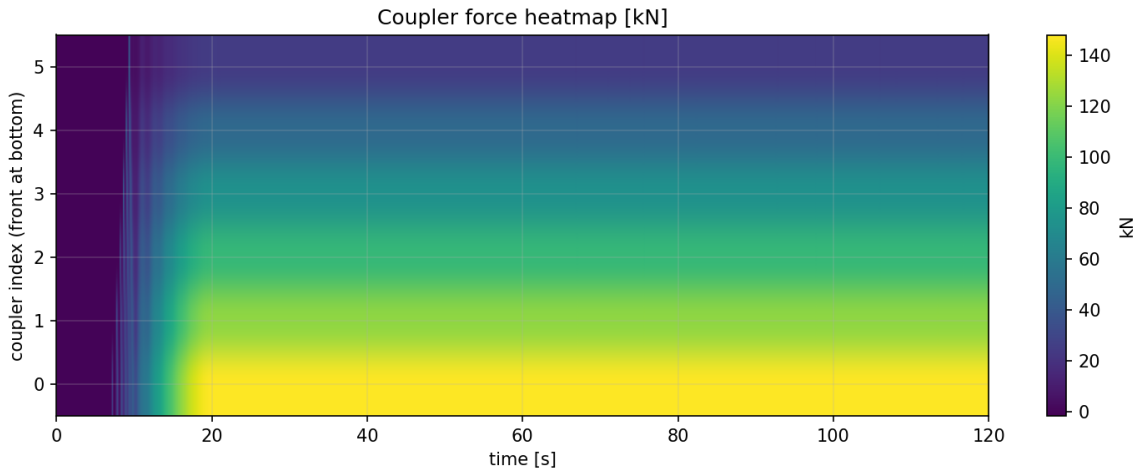


Figure 3.7: Stage 5: coupler force heatmap (force versus time and coupler index).

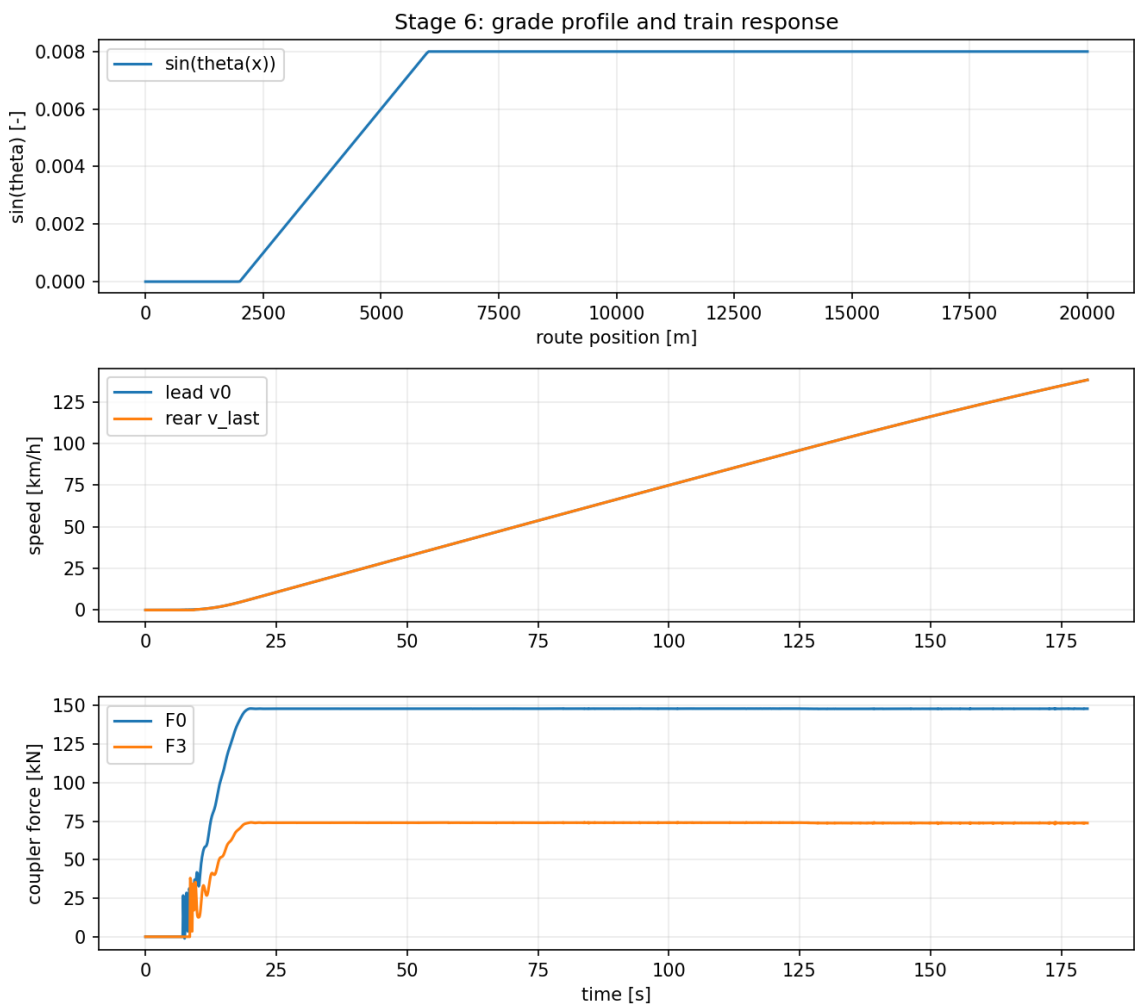


Figure 3.8: Stage 6: interpolated grade profile $\sin\theta$ versus route position, with lead and rear speeds and coupler forces versus simulation time.

### 3.2.7 Stage 7: actuator dynamics and power-limited traction

The state is augmented by first-order lag variables for commanded brake level $z_{\mathrm{brk},i}$ and commanded traction level $z_{\mathrm{trac},i}$ on each vehicle, driven by exogenous commands $u_{\mathrm{brk},i}(t)$ and $u_{\mathrm{trac},i}(t)$:

$$\dot{z}_{\mathrm{brk},i} = \frac{u_{\mathrm{brk},i} - z_{\mathrm{brk},i}}{\tau_b}, \qquad \dot{z}_{\mathrm{trac},i} = \frac{u_{\mathrm{trac},i} - z_{\mathrm{trac},i}}{\tau_t}. \tag{3.14}$$

Achieved brake force is capped by a vehicle limit, $F_{\mathrm{brk},i} = \min(\max(z_{\mathrm{brk},i}, 0), F_{\mathrm{brk,max},i})$. For powered vehicles, achieved traction is the continuous power cap

$$F_{\mathrm{trac},i} = \min\left(\max(z_{\mathrm{trac},i}, 0),\ \frac{P_{\max}}{\max(|v_i|, \varepsilon)}\right), \tag{3.15}$$

with $P_{\max}$ an illustrative constant power ceiling and $\varepsilon$ a small speed regulariser. These substitutes replace instantaneous step forces in eq. (3.13). Figure 3.9 shows lead speed together with the lead's $z_{\mathrm{brk}}$ and $z_{\mathrm{trac}}$ for a scenario that overlaps traction command with braking command; the plotted states should not be confused with achieved $F_{\mathrm{trac},i}$ and $F_{\mathrm{brk},i}$, which also reflect eq. (3.15) and the brake saturation. The notebook uses this stage to motivate realistic control shaping; a full driver model or anti-slip logic is outside the present scope.

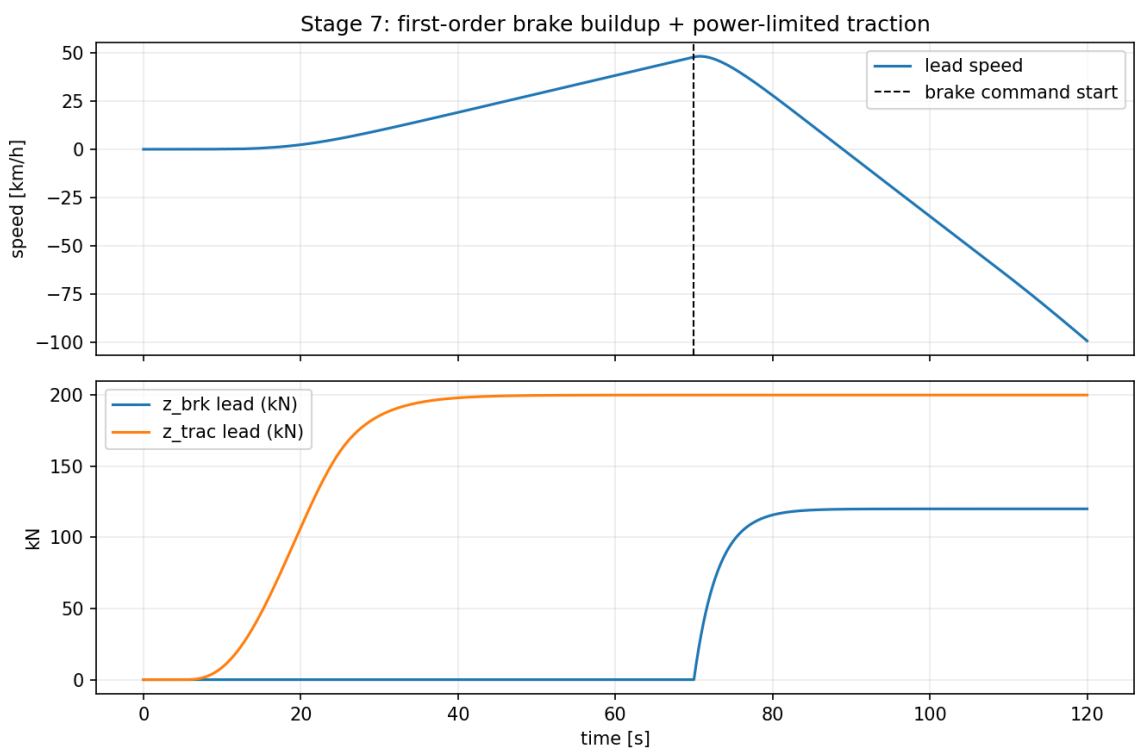


Figure 3.9: Stage 7: lead speed with brake-command onset marker; first-order lag states $z_{\mathrm{brk}}$ and $z_{\mathrm{trac}}$ on the lead versus time.

### 3.2.8 Stage 8: scope relative to industrial high fidelity

The notebook closes the incremental build with a structured discussion of effects commonly associated with "high fidelity" in operational or research simulators: hysteretic draft gear, pneumatic brake pipe dynamics, distributed traction and adhesion limits, stochastic parameters, and calibration to field data. Those modules are not embedded in the reference code path summarised here; they delimit the boundary between the intermediate simulator used for learning experiments and a fully industrial toolchain. Packaging the working right-hand side behind a small simulation API (subsequent notebook stage) is deferred to the architecture chapter where the surrogate is coupled to experiments.

## 3.3 Input definitions

### 3.3.1 Route profile

A route profile $\mathcal{R}$ defines the track geometry along the path from origin to destination. We represent $\mathcal{R}$ as a sequence of segments $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$, where each segment $s_j$ is characterized by:

- Grade $g_j$: elevation change per unit distance (%)
- Curvature $\kappa_j$: track curvature (1/m)
- Speed limit $v_{\text{lim},j}$: maximum allowed speed (m/s)
- Segment length $\Delta s_j$: distance covered (m)

### 3.3.2 Consist configuration

A consist $\mathcal{C} = \{c_1, c_2, \ldots, c_m\}$ is an ordered set of vehicles, where $c_1$ is the lead locomotive and $c_i$ for $i > 1$ are trailing cars. Each vehicle $c_i$ has properties:

- Mass $m_i$ (kg)
- Length $L_i$ (m)
- Resistance coefficients (Davis equation parameters)
- Coupler type and capacity

### 3.3.3 Control plan

A control plan $\mathcal{U}$ specifies the traction/braking commands as a function of time or position. We represent $\mathcal{U}$ as a sequence $\mathbf{u} = [u(t_1), u(t_2), \ldots, u(t_T)]$, where $u(t)$ is the net force command at time $t$ (normalized to $[-1, 1]$ for full braking to full traction).

## 3.4 Output definitions

We distinguish three categories of outputs: trajectory-level quantities (how the train state evolves over time), aggregate quantities (single numbers summarizing the run), and risk metrics (probabilistic safety measures).

### 3.4.1 Trajectory-level outputs

The simulator produces time histories of longitudinal state and internal forces. In classical form, one may write a stacked trajectory $x(t)$ over $t \in [0, T]$; in the tensorized layout of section 3.1.3, the same information is represented by $\mathbf{H}(t)$ and $\mathbf{E}(t)$, or by discrete rollouts $\mathbf{X}_{k,:,:}$ and $\mathbf{E}_{\text{hist},k,:,:}$ at sample times $t_k$. Positions, velocities, actuator states, and static attributes appear in $\mathbf{H}$; coupler extensions, relative speeds, forces, and coupler parameters appear in $\mathbf{E}$. Accelerations follow by differentiation or from the force balance if needed for post-processing. The coupler force in the $i$th interface (force on the rear vehicle of the pair) is one channel of $\mathbf{E}(t)$ and can also be written as a functional of the full state,

$$F_{\text{c},i}(t) = f_i(\mathcal{R}, \mathcal{C}, \mathcal{U}, t), \tag{3.16}$$

where $f_i$ denotes the map implied by the physics and constitutive laws (section 3.2).

### 3.4.2 Aggregate outputs

Given inputs $(\mathcal{R}, \mathcal{C}, \mathcal{U})$, the simulator also produces scalar summaries:

$$E = \int_0^T P(t)\, dt \tag{3.17}$$

$$T = T \tag{3.18}$$

where $P(t)$ is instantaneous power consumption.

### 3.4.3 Risk metrics

Beyond point values, we care about the probability that safety constraints are violated. For a threshold $\tau$ (e.g., coupler capacity), we define the exceedance probability:

$$\mathbb{P}(F_{\text{c},i}(t) > \tau) = \mathbb{P}(F_{\text{c},i}(t) > \tau) \tag{3.19}$$

We also predict quantiles $Q_p$ such that $\mathbb{P}(Y \leq Q_p) = p$ for $p \in \{0.05, 0.25, 0.50, 0.75, 0.95\}$.

## 3.5 Uncertainty sources

Uncertainty in train dynamics predictions arises from:

1. **Model uncertainty**: Incomplete physics (simplified resistance models, unknown friction coefficients)

2. **Parameter uncertainty**: Mass estimates, coupler stiffness, brake effectiveness
3. **Environmental uncertainty**: Wind, temperature effects on adhesion
4. **Operational uncertainty**: Driver behavior, signal delays

### 3.5.1 Formal Treatment of Uncertainty

To make the probabilistic formulation rigorous, we introduce a random vector $\xi$ representing all uncertain factors (mass errors, friction variation, environmental conditions, etc.). We write $\xi \sim p(\xi)$ to denote that $\xi$ is drawn from a distribution $p$. The simulator output then depends on $\xi$:

$$\mathbf{y} = F(\mathcal{R}, \mathcal{C}, \mathcal{U}; \xi), \tag{3.20}$$

consistent with the deterministic output $\mathbf{y}$ in eq. (3.4). For fixed inputs $(\mathcal{R}, \mathcal{C}, \mathcal{U})$, different realizations of $\xi$ yield different outputs. The exceedance probability is now defined precisely as the probability over $\xi$:

$$\mathbb{P}_\xi(F_{\mathrm{c},i}(t) > \tau) = \int_{\{\xi : F_{\mathrm{c},i}(t;\xi) > \tau\}} p(\xi)\, d\xi \tag{3.21}$$

## 3.6 Supervised Learning Objective

Given a dataset $\mathcal{D} = \{(\mathcal{R}^{(k)}, \mathcal{C}^{(k)}, \mathcal{U}^{(k)}, \mathbf{y}^{(k)})\}_{k=1}^{N}$ where $\mathbf{y}^{(k)}$ contains ground truth outputs from high-fidelity simulation, we seek a surrogate model $f_\theta$ that minimizes:

$$\mathcal{L}(\theta) = \sum_{k=1}^{N} \left[ \mathcal{L}_{\text{quantile}}(\theta; \mathbf{y}^{(k)}) + \lambda_{\text{cal}} \mathcal{L}_{\text{cal}}(\theta; \mathbf{y}^{(k)}) + \lambda_{\text{phys}} \mathcal{L}_{\text{phys}}(\theta) \right] \tag{3.22}$$

where $\mathcal{L}_{\text{quantile}}$ is the quantile regression loss, $\mathcal{L}_{\text{cal}}$ encourages calibration, and $\mathcal{L}_{\text{phys}}$ enforces physics consistency.

### 3.6.1 Loss Term Definitions

**Quantile regression loss.** For a target variable $Y$ and predicted quantile $\hat{Q}_p$ at probability $p$, the pinball (quantile) loss is

$$\rho_p(y, \hat{q}) = \begin{cases} p(y - \hat{q}) & \text{if } y \geq \hat{q} \\ (1-p)(\hat{q} - y) & \text{if } y < \hat{q} \end{cases} \tag{3.23}$$

The quantile loss $\mathcal{L}_{\text{quantile}}$ sums $\rho_p$ over all quantile levels and output variables.

**Calibration loss.** A model is *calibrated* if, among predictions with nominal confidence $p$, the true fraction of outcomes falling within the predicted interval is approximately $p$. $\mathcal{L}_{\text{cal}}$ penalizes deviations from this property.

**Physics regularization.** $\mathcal{L}_{\text{phys}}$ encourages the surrogate to satisfy known physical constraints (e.g., energy balance, force equilibrium).

## 3.7 Conceptual Pipeline

The overall workflow can be summarized as a pipeline:

1. **Inputs**: Route, consist, and control plan are specified.
2. **High-fidelity simulation**: The deterministic operator $F$ (or its probabilistic counterpart with $\xi$) is evaluated to produce outputs.
3. **Dataset**: Many such runs are collected, forming $\mathcal{D}$.
4. **Surrogate operator**: A neural network $f_\theta$ is trained to approximate $F$ (and its probabilistic outputs).
5. **Probabilistic outputs**: At inference time, the surrogate produces quantiles, exceedance probabilities, and point predictions without running the expensive simulator.

Figure 3.10: Conceptual pipeline from structured inputs to probabilistic outputs via high-fidelity simulation and surrogate modeling.

## 3.8 Evaluation Targets

We evaluate the surrogate model on four dimensions:

**Accuracy.** Mean absolute error (MAE) and root mean squared error (RMSE) for deterministic outputs (energy, arrival time).

**Calibration.** Expected calibration error (ECE), reliability diagrams. A model is well-calibrated if stated confidence intervals match empirical coverage.

**Safety.** Constraint violation rates; alignment of exceedance probabilities with empirical rates.

**Efficiency.** Inference time compared to high-fidelity simulation, measured as speedup factor.

## 3.9 Chapter summary

This chapter formulated the freight-train forecasting problem and the operator-learning setting built on top of it. Classical longitudinal dynamics provides the physical foundation (eq. (3.1)); in this thesis the same system is recast into node and edge tensors $\mathbf{H}(t)$ and $\mathbf{E}(t)$ so that simulation rollouts align with structured sequence models and with probabilistic learning targets (sections 3.1.3 and 3.1.4). The simulator $F$ maps structured inputs $(\mathcal{R}, \mathcal{C}, \mathcal{U})$ to structured outputs $\mathbf{y}$: trajectory-level tensors (or equivalent stacked states and coupler forces), aggregate quantities such as energy and trip time, and risk metrics including exceedance probabilities and quantiles (eq. (3.4)). We distinguished deterministic outputs under fixed parameters from outputs under random $\xi$ (eq. (3.20)).

The learning task is difficult because (1) the input space is high-dimensional and structured, (2) outputs span multiple scales and types, (3) uncertainty quantification is required for safety-relevant decisions, and (4) physical consistency must be preserved. Uncertainty is handled by modeling $\xi \sim p(\xi)$ and training the surrogate to predict quantiles and exceedance probabilities with calibration.

A reference LTD model documented stage by stage in section 3.2 supplies reproducible ground truth for experiments. The next chapter introduces the framework architecture: a hybrid physics-aware sequence model that approximates $F$ while respecting constraints and providing calibrated probabilistic outputs compatible with the tensorized state layout.